

编译原理作业1：熟悉Clang编译器

本作业旨在熟悉Clang编译器的安装、运行和调试，并通过一个包含多种特性的C程序，观察和分析Clang编译器在各个逻辑阶段（预处理、词法分析、语法分析、中间代码生成、汇编代码生成、目标文件生成、链接执行）的输出结果。

1. 实验环境与安装

- 操作系统: Windows
- 环境配置: MSYS2 (MinGW-w64 x86_64)
- 编译器: Clang 21.1.5 (x86_64-w64-windows-gnu)
- 安装命令: `pacman -S mingw-w64-clang-x86_64-clang`

2. 实验用的C程序

编写了一个包含宏定义、全局变量、函数声明与调用、条件语句和循环语句的C程序 `example.c`：

```
1  #include <stdio.h>
2
3  // 宏定义 - 预处理阶段会展开
4  #define MAX_SIZE 100
5  #define SQUARE(x) ((x) * (x))
6
7  // 全局变量
8  int globalVar = 42;
9
10 // 函数声明
11 int add(int a, int b);
12 void printMessage(const char* msg);
13
14 // 主函数
15 int main() {
16     int x = 10;
17     int y = 20;
18     int result;
19
20     // 使用宏
21     int arr[MAX_SIZE];
22     int squared = SQUARE(5);
23
24     // 函数调用
25     result = add(x, y);
26
27     // 条件语句
28     if (result > 25) {
29         printMessage("Result is greater than 25");
30     } else {
31         printMessage("Result is 25 or less");
32     }
33
34     // 循环语句
35     for (int i = 0; i < 5; i++) {
```

```

36     arr[i] = i * 2;
37 }
38
39 printf("x = %d, y = %d, result = %d\n", x, y, result);
40 printf("squared = %d, globalVar = %d\n", squared, globalVar);
41
42 return 0;
43 }
44
45 // 函数定义
46 int add(int a, int b) {
47     return a + b;
48 }
49
50 void printMessage(const char* msg) {
51     printf("%s\n", msg);
52 }

```

3. 编译器的各个逻辑阶段及其输出结果

Clang作为一个基于LLVM的前端，其编译过程主要分为前端（词法、语法分析）、中端（IR优化）、后端（代码生成）。下面展示了各阶段的命令和输出分析。

3.1 预处理阶段 (Preprocessing)

命令: `clang -E example.c`

过程理解: 处理所有的预处理指令，如 `#include` 导入头文件内容，`#define` 展开宏替换。在这个阶段，`SQUARE(5)` 会被替换为 `((5) * (5))`，`MAX_SIZE` 会被替换为 `100`。还会去除注释。

输出特征: 输出了展开后的巨长代码（超过26000行），主要包含了 `stdio.h` 等标准库的内容，然后在文件的最末尾才出现我们自己的被展开的C代码。

3.2 词法分析阶段 (Lexical Analysis)

命令: `clang -xclang -dump-tokens example.c`

过程理解: 将字符流分解成有意义的词法单元 (Token)，如关键字 (keyword)、标识符 (identifier)、常量 (constant)、运算符 (operator) 等。

部分输出结果:

```

1 void 'void' [StartOfLine] Loc=<D:/msys64/clang64/include/_mingw.h:604:1>
2 __attribute '__attribute__' [LeadingSpace] Loc=
  <D:/msys64/clang64/include/_mingw.h:604:6>
3 l_paren '(' Loc=<D:/msys64/clang64/include/_mingw.h:604:6>
4 identifier '__cdecl__' Loc=<D:/msys64/clang64/include/_mingw.h:604:6>
5 ...
6 int 'int' [LeadingSpace] Loc=<D:/msys64/clang64/include/_mingw.h:625:58>
7 identifier 'code' [LeadingSpace] Loc=
  <D:/msys64/clang64/include/_mingw.h:625:62>
8 r_paren ')' Loc=<D:/msys64/clang64/include/_mingw.h:625:66>
9 semi ';' Loc=<D:/msys64/clang64/include/_mingw.h:625:67>

```

可以看到，代码被切分成了像 `int`, `identifier`, `l_paren` (左括号) 这样的基本单元，并记录了它们在源代码中的位置(Loc)。

3.3 语法分析阶段 (Syntax Analysis)

命令: `clang -xclang -ast-dump example.c`

过程理解: 将词法分析生成的Token流转换为抽象语法树 (AST)。这个阶段会检查语法是否正确，并体现程序的层次结构。

部分输出结果:

```
1 TranslationUnitDecl 0x1b98a1890c8 <<invalid sloc>> <invalid sloc>
2 | -TypedefDecl 0x1b98a189a28 <<invalid sloc>> <invalid sloc> implicit
  __int128_t '__int128'
3 | ` -BuiltinType 0x1b98a189690 '__int128'
4 | -FunctionDecl 0x1b98a205d28 <D:/msys64/clang64/include/_mingw.h:604:1,
  col:31> col:14 __debugbreak 'void (void) __attribute__((cdecl))': 'void
  (void)'
5 ...
6 | -FunctionDecl 0x1b98a206458 prev 0x1b98a206290 <line:90:31, line:640:1>
  line:626:60 __fastfail 'void (unsigned int) __attribute__((noreturn))' extern
  inline
7 | | -ParmVarDecl 0x1b98a206358 <col:71, col:84> col:84 used code 'unsigned
  int'
8 | | -CompoundStmt 0x1b98a2068e0 <line:627:1, line:640:1>
9 | | | -GCCAsmStmt 0x1b98a206678 <line:632:3, col:48>
10 | | | | -ImplicitCastExpr 0x1b98a206660 <col:43> 'unsigned int'
  <LValueToRValue>
11 | | | | ` -DeclRefExpr 0x1b98a206640 <col:43> 'unsigned int' lvalue ParmVar
  0x1b98a206358 'code' 'unsigned int'
```

抽象语法树中包含了如 `FunctionDecl` (函数声明)、`ParmVarDecl` (参数变量声明)、`CompoundStmt` (复合语句) 等节点，展示了代码的作用域和控制流。

3.4 中间代码生成阶段 (IR Generation)

命令: `clang -S -emit-llvm example.c -o example.ll`

过程理解: 将AST转换为LLVM的中间表示 (IR)。LLVM IR是一种平台无关的、强类型的类似汇编的语言。

部分输出结果:

```
1 @globalVar = dso_local global i32 42, align 4
2 @.str = private unnamed_addr constant [26 x i8] c"Result is greater than
  25\00", align 1
3 @.str.1 = private unnamed_addr constant [21 x i8] c"Result is 25 or less\00",
  align 1
4
5 ; Function Attrs: noinline nounwind optnone uwtable
6 define dso_local i32 @main() #0 {
7     %1 = alloca i32, align 4
8     %2 = alloca i32, align 4
9     %3 = alloca i32, align 4
```

```

10     store i32 10, ptr %2, align 4
11     store i32 20, ptr %3, align 4
12     %8 = load i32, ptr %2, align 4
13     %9 = load i32, ptr %3, align 4
14     %10 = call i32 @add(i32 noundef %8, i32 noundef %9)
15     store i32 %10, ptr %4, align 4
16     %11 = load i32, ptr %4, align 4
17     %12 = icmp sgt i32 %11, 25
18     br i1 %12, label %13, label %14
19     ...
20 }

```

可以看到，局部变量通过 `alloca` 指令在栈上分配内存（以 `%` 开头的寄存器），然后使用 `load` 和 `store` 进行读写，函数通过 `call` 调用，条件分支通过 `icmp` 和 `br` 实现。

3.5 汇编代码生成阶段 (Assembly Generation)

命令: `clang -S example.c -o example.s`

过程理解: LLVM后端将优化后的IR代码转换为特定目标机器（这里是x86_64）的汇编代码。

部分输出结果:

```

1  main:                                     # @main
2  .seh_proc main
3  # %bb.0:
4      pushq    %rbp
5      subq     $464, %rsp                    # imm = 0x1D0
6      leaq     128(%rsp), %rbp
7      callq    __main
8      movl     $10, 328(%rbp)
9      movl     $20, 324(%rbp)
10     movl     $25, -84(%rbp)
11     movl     328(%rbp), %ecx
12     movl     324(%rbp), %edx
13     callq    add
14     movl     %eax, 320(%rbp)
15     cmpl     $25, 320(%rbp)
16     jle     .LBB0_2

```

这是标准的x86_64汇编语言，利用了具体的寄存器如 `%rbp`、`%rsp`、`%ecx`、`%edx` 等，以及指令如 `pushq`、`movl`、`callq` 等。

3.6 汇编器和链接器阶段

- **目标文件生成 (Assembler):** `clang -c example.c -o example.o`
将汇编代码转换为机器可以直接执行的二进制指令（对象文件）。生成的 `example.o` 是 x86-64 COFF object file 格式。
- **链接与执行 (Linker):** `clang example.c -o example.exe`
将各个目标文件及所需的标准库文件链接在一起，生成最终的可执行文件。

运行生成的 `./example.exe` 输出：

```
1 | Result is greater than 25
2 | x = 10, y = 20, result = 30
3 | squared = 25, globalVar = 42
```

4. 总结与体会

通过以上各个阶段的操作，可以清晰地观察到一段C代码是如何被编译器逐步降解的：

1. **预处理**扩展代码，解决宏和库引入。
2. **词法与语法分析**（Clang前端）验证代码合法性，建立树状逻辑结构（AST）。
3. **IR生成**将其标准化为一种通用的中间表示（LLVM IR）。这一步是LLVM架构设计的精髓，使得前端与后端解耦。
4. **后端与链接**则将通用表示转化为针对特定硬件的机器码并组装成可执行文件。